

Guiding Global Software Development Projects using Scrum and Agile with Quality Assurance

Christelle Scharff
Pace University, New York, NY, USA
cscharff@pace.edu

Abstract

This paper focuses on a global software development project where extended teams of students distributed across two to three countries, namely the US, Cambodia, India and Senegal, experienced the roles of developers, auditors and testers. Developers used Scrum and Agile to develop mobile applications for different mobile platforms with the support of different end-to-end tooling infrastructures. This paper isolates and focuses on the role of auditors. It describes the model of collaboration, the role of auditing in Agile and Scrum adherence, and the importance of tools to support quality assurance activities. Recommendations for a better involvement of auditors in Agile and Scrum projects and the expected benefits of their contribution are discussed.

1. Introduction

Global software development (GSD) has become a standard and common practice of the industry. Many research have highlighted its latent benefits from reducing cost to improving software quality and inherent challenges linked with distance, culture and time difference [1,7]. Various innovative initiatives have focused on reflecting the reality of global software development in the classroom. A more recent paradigm shift of the software industry is the advent and success of the rapidly evolving mobile technology field. The latest predictions from Morgan Stanley stated “Forget the desktop, the future is mobile all the way” (April 2010). Powerful and affordable mobile devices are coming out every day. The abounding number of online stores for mobile applications allows developers and entrepreneurs to enter this promising field. Consequently, even without mentioning the potentials of the Mobile Web and SMS, skills in mobile application development are becoming a high demand commodity for future IT professionals.

Since 2005 we have been running an annual global software development project that investigates different models for students distributed across three to four countries to develop software together [5,6,7,10]. The emphasis evolved on different themes each year, ranging from global supply chains, tooling, software quality assurance, integration, deployment and maintenance, entrepreneurship, socialization, mobile application development, and distributed teams of developers. Relevant to the present paper, we examined models comprising: 1) coaches and auditors to improve software quality and likelihood of deployment on project using a loose waterfall process (2007 & 2008); 2) mobile application development to introduce students to this cutting-edge field (2009); and 3) end-to-end tooling infrastructures to support developers following Agile and Scrum to transition to a sustainable use of these practices (2009).

This paper focuses on the 2010 global software development project where extended teams of students distributed across two to three countries at different places of the global software development equation, namely the US, Cambodia, India and Senegal, experienced the roles of developers, auditors and testers. Students developed mobile applications using Agile and Scrum for different platforms with the support different end-to-end tooling infrastructures. The targeted mobile platforms were Android, Blackberry and Java ME to capitalize on the

existing Java skills of the students. The end-to-end tooling solutions, specifically IBM Rational Team Concert (RTC), Rally Software, and Redmine, were used to simplify and minimize the numbers of tools for engineering, communication, and project management on the project. The project started with four weeks of preparation followed by eight weeks of development. The wiki of the project is available at [12].

The study we describe in this paper isolates and focuses on the role of auditors on projects using Agile and Scrum. The role of testers is part of a parallel study. Taking different perspectives, Weinberg, Juran and Crosby defined software quality as “value to some people”, “fitness for use”, and “conformance to requirements”. SQA includes activities such as requirements, design and code reviews, requirements and defect tracking, and testing, to achieve software quality. Auditing is often a regulatory activity undertaken internally or externally to check compliance. It may include interviews and reviews of documentary evidence. The rule in auditing is about “Say what you do; Do what you say; Prove it”. In addition, auditing can be used to alert developers that actions need to be taken to improve effectiveness before it is too late. Introduced in 2007 and 2008, we re-instated the auditor role in 2010 but this time on projects following Agile and Scrum. The goal was to improve not only the quality of the developed software but also to augment Agile and Scrum adherence. Auditors added an external eye on quality on the project, while the Scrum Master has an internal day-to-day role. Three end-to-end tools were used by the teams to evaluate their strengths and weaknesses and investigate how well they support quality assurance activities in our setting.

Our study attempted to examine the following research questions:

- Software Quality Assurance -- How well do quality assurance activities focusing on audits guide and augment adherence to Agile and Scrum?
- Role of the Tooling -- How important is tooling in supporting quality assurance activities in a distributed setting where developers are using Agile and Scrum?
- Recommendations -- How to integrate and what are the benefits of quality assurance activities in a distributed setting where developers are using Agile and Scrum?

The paper is organized as follows. In Section 2, we describe the 2010 project scenario. Section 3 shows how we adapted and supported Agile and Scrum practices as well as quality assurance activities in our context. Section 4 highlights our findings, and discusses the impact of the audits on quality and process adherence and the related role of tools. Section 5 concludes the paper and presents our future work.

2. Project scenario

In this section, we describe the details of the 2010 GSD project. We present the learning objectives, the teams and the roles of the team members, the mobile applications that were developed, the process that was followed and the tooling that was adopted.

2.1. Learning objectives

The goal of the project was to prepare different groups of students with key skills for the global and mobile labor market. Students were provided with a real global software development experience where they played different roles, used different tools, and were part of large teams distributed across two to three countries. Developers and testers had the opportunity to gain skills in mobile application development for either Android, Blackberry or Java ME platforms. Usability testing was emphasized as usability is a key factor in mobile application adoption [2].

2.2. Teams

The 2010 GSD project involved 43 students – 9 undergraduate students of the Capstone Software Engineering course at Pace University in the US, (<http://pace.edu>) 19 graduate students of the Software Quality Assurance & Reliability course of the Master of Software Engineering and Design at Pace University, and 15 students from institutions in India, Senegal and Cambodia who volunteered to be part of this project in addition to their school and internship commitments. Out of these 15 students, 2 undergraduate students were from the Royal University of Phnom Penh in Cambodia (<http://rupp.edu.kh>), 2 graduate students were from the University of Delhi in India (<http://du.ac.in>), 3 graduate students were from the Ecole Supérieure Polytechnique in Dakar, Senegal (<http://esp.sn>), 3 students were from the Ecole Supérieure Multinationale des Télécommunications in Dakar, Senegal (<http://esmt.sn>), and 5 undergraduate students were from the Université de Thiès in Thiès, Senegal (<http://univ-thies.sn>). Students in India, Cambodia and Senegal were motivated in participating in a project involving mobile technology, Agile and Scrum practices, and global software development. Senegalese students were also interested in improving their English skills. All students wanted to build their resumes with an international experience. The students of Thiès and ESP had experience with mobile technology through a Java ME course taught by the US instructor. The two Indian students participated in the 2009 GSD project as developers. The 4 instructor coaches who participated in the project - 1 in the US, 1 in India and 2 in Senegal (ESMT) - have been working together on different projects including the annual global software development project. They were responsible in monitoring the project and engaging the students at their different universities. Governance was managed from the US and students at the different locations were in contact with the US faculty. We put in place a model of working that accommodated all the parties, both instructors and students, without compromising the learning objectives of the different courses and interests of the students who volunteered on the project. The 43 students were distributed in three extended teams of 11, 12 and 20 students. In each team, students undertook the roles of developers (US undergraduate students), auditors (US graduate students) and testers (students in Cambodia, India and Senegal). The extended teams were comprised of students from at least two or three countries and time zones.

2.3. Software projects

Students were asked to choose a mobile platform amongst Android, Blackberry and Java ME. They proposed to develop mobile applications on the theme of Improving Students' Life after researching on existing solutions and potential competitors. The following three mobile applications were developed by the three teams.

- Godiva Flash Cards is an Android flash cards application where users can create flash cards with questions and answers. Flash cards can have images and be organized in sets. The cards can be searched, viewed, shared with friends, edited, and rated by the community. They can be marked as right and wrong during evaluation. The originality of Godiva Flash Cards is its social aspect based on sharing and communal editing which is not common in current flash cards applications on the Android Market.
- NoInk is a Blackberry application that students can use to take notes in the form of pictures (from the board in the classroom for example). The notes can be saved, deleted, annotated, and organized by topics to become searchable. The originality of NoInk is technological as the code was written using the Rhomobile framework, an open source and cross-platform mobile application framework written in Ruby. The developers of NoInk contributed to the Rhomobile project.

- BackPocket is a Java ME application that runs on feature phones and permits students to manage their finances and control their spending on the go. Students can record their expenses and assign them to categories. The dates of the spending are automatically saved. Spending can be visualized as pie charts. There is quite a number of budgeting applications available for Java ME phones but this application is the only one focusing on students' finances.

2.4. Process

Since 2005, teams have been following a loose waterfall process with iteration and feedback. Starting in 2009, we switched to the use of Agile and Scrum in the global software development project. From an instructor point of view, the key advantage of using Agile and Scrum is the increased visibility in the regular accomplishments and progress of the students. When using waterfall, coding began on the ninth week of the project and ended in the fourteenth week, whereas with Agile and Scrum a working software is demonstrated at the end of each sprint. In 2010 students who played the roles of the developers and auditors followed lectures on process and played the XP game in class to assimilate the practices of Agile and Scrum. The students in Cambodia, India and Senegal were assigned readings and videos on Agile and Scrum to get enough understanding of the software engineering process of the project.

2.5. Tooling

The 2009 version of the project was the first attempt to reduce the number of tools to be used in global software development projects for engineering, communication, project management and socialization. It converged to the use of IBM Rational Team Concert (RTC) as an end-to-end tooling infrastructure (<http://jazz.net>) built on Eclipse, an integrated development environment familiar to students. In 2010 we choose to experiment with other tools to have a better understanding of the strengths and weaknesses of each of them and investigate how they support global software development and quality activities such as auditing and testing. During the four first weeks of the project students were provided with tutorials on the tools they were to use in the project. Each team used a different tooling infrastructure – RTC, Rally Software (<http://rallydev.com>) and Redmine (<http://redmine.org>). RTC and Rally support Agile and Scrum practices, whereas Redmine is more of a generic project management tool. Developers choose Redmine because of their familiarity with the tool but it does not offer specific support for Agile and Scrum. RTC is an integrated environment offering support for agile planning, source code management (RTC Source Control), and work items classified as user stories, tasks and bugs. Communication and coordination are facilitated through features directly available in RTC, emails and chats for asynchronous and synchronous communications, wikis for planning and for sharing artifacts, and a change notification mechanism to increase team awareness. Diverse charts are available to assess the project's health including burn down charts. A web UI is available for non-technical roles on the project. Rally Software permits to organize user stories, tasks, tests and bugs. It offers capabilities to support planning and iterations. The dashboard can be customized and include views from other tools (e.g., Google Groups and Google Code). A large number of reports are available (e.g., printouts of the Backlog user stories and burn down charts). Table 1 summarizes the different tools used in the projects organized by categories.

Table 1. Tools used by the different teams in GSD 2010.

	Team 1 - Android Team	Team 2 - Blackberry Team	Team 3 - Java ME Team
Development	Eclipse, Android SDK and Android plug-in Java and XML Mercurial and Google Code	Rhomobile and Blackberry simulator Ruby, HTML, CSS Git	Eclipse, Sun Wireless toolkit, and Java ME plug-in Java ME RTC Source Control
Project Management	Rally Google Calendar	Redmine Google Calendar	RTC Google Calendar
Communications	Google Groups	Anologue Chat Google Groups Wikis in Redmine	Google Groups Wikis in RTC
Audits	Google Docs referenced in the Google Groups	Google Docs referenced in the wikis in Redmine	Wiki pages in RTC
Testing	Rally for bug submissions Word for reports	Redmine for bug submissions Word for reports	RTC for bug submissions Word for reports

3. Quality assurance on Agile and Scrum projects

In this section, we describe how we adapted and supported Agile and Scrum practices for the development of the targeted mobile applications and what quality assurance activities were undertaken in the 2010 GSD project.

3.1. Agile and Scrum implementation

Agile methodologies were defined in the Agile Manifesto as methodologies that value individuals and interactions, working software, customer collaboration, and responding to changes [8]. They acknowledge the fact that requirements change and recognize that a good relationship between the client and the developers is crucial for success in software development. They cover the engineering disciplines of software engineering such as requirements, analysis and design, coding and testing, activities that are commonly performed all together in iterations that end with shippable software. All the practices of particular methodologies cannot be adopted from the start and without experience. In this study where mobile applications were developed students were encouraged to enforce the following principles augmented by some mobile application development specifics:

- The client must be constantly involved in the process.
- Requirements must be captured at a high level.
- The team must accept the reality of requirements changes.
- Requirements must be prioritized and the 80/20 rule must be applied, i.e., 80% of the time will be spent on 20% of the features with more priority to the client.
- Requirements will include acceptance test that are written and verified by the client.
- A high level and visual design of the mobile application to be developed must be produced and it must integrate the best user interface design practices of the targeted platform.
- Coding will be done in pairs [13].
- Code will implement the Model View Controller design pattern (MVC), follow a standard coding style, be shared in a centralized repository, and be refactored by team members.
- Testing will be integrated in the project lifecycle and done early.

Scrum is a generic framework that instantiates the project management part of the software engineering process and makes abstraction of the engineering part. It is commonly used in

combination with Agile [4,11]. Scrum uses ceremonial roles and meetings. Iterations of fixed durations are called Sprints. The Product Owner is responsible for achieving business value and in charge of the Product Backlog, a set of requirements written as prioritized user stories with business values. The developers constitute the Scrum Team. The Scrum Master guides the team through Scrum and helps to resolve issues. During the Sprint Planning, the team commits to complete a certain number of tasks that are estimated and reported in a Sprint Backlog. At the end the Sprint, a Sprint Demo and a Sprint Retrospective take place to demo the software and reflect on the sprint. The velocity (business value achieved) and morale of the teams and the quality of the work are evaluated. The team meets in Daily Standup Scrum Meetings where each team member responds to three questions: (1) What did you do since the last meeting?; (2) What will you do until the next meeting?, and (3) What are the impediments you encounter? The Scrum Master maintains a Sprint Burn Down Chart as a metrics for the team's progress in terms of work remaining. In this study students were encouraged to enforce the following Scrum principles:

- The team, which includes the customer, will communicate regularly to share information and create visibility. It will work toward a common and fixed goal.
- The team will work in fixed iteration where the work will be planned, prioritized and committed based on the difficulty of the work and the value to the customer. Value has to be delivered early to the customer.
- The team will be empowered to take control of its destiny and reach its full potential. It will make decisions based on the work achieved and the common goal.
- The team will learn and improve during the project by looking at the outcomes of the different iterations and reflecting on the actions that need to be taken to improve the results.

3.2. Scrum characteristics

Timeline of the project. The project spanned fourteen weeks with four weeks for training, research, project proposal and the creation of the initial product backlog, four sprints of two weeks, one week vacation between sprints 2 and 3, and one week for the final presentations in front of university representatives at Pace University.

Scrum roles. The Scrum Team was composed of the developers in the US. One of the developers played the role of the Product Owner in addition to the developer role. We thought that it was suitable for the instructor in the US, who is also a professional Certified Scrum, to play the role of Scrum Master. Indeed, the role of instructor and Scrum Master interleaves often. The use of Agile and Scrum in educational setting requires instructors to be vigilant at all time, the cycle of the work being bound with the length of a sprint.

Scrum ceremonies. The "Daily" Scrum Meetings took place twice a week during class time and were orchestrated by the Scrum Master. Developers answered the three traditional questions of Scrum that were adapted to our context: (1) What did you do since our last meeting? (2) What will you do until our next meeting? and (3) What are the problems that you encountered? They reported their answers in writing in their project management tools for future analysis and to increase visibility for the quality team. The Sprint Planning Meetings took place in class and were finalized outside of class. The Sprint Goal was clearly stated in the project management tools to increase awareness and commitment. Developers determined the user stories they would implement during the sprint in conjunction with the Product Owner and decomposed the targeted stories into tasks. The Sprint Demos and the Sprint Retrospectives occurred in class at the end of each sprint. The students were asked to prepare videos of their software before the meetings and make them available online. They were also to prepare the Sprint Retrospectives by posting what worked well and what could

have worked better during the sprint in their project management tools. They had to clearly state the stories they committed to implement during the Sprint, the actual stories they implemented during the Sprint, and their estimated and actual velocities. They also specified the problems they met that were related to Scrum. The auditors had access to the notes of the Scrum Meetings, Planning Meetings, Retrospectives and Demos to perform audits.

Scrum artifacts. The Product Backlogs and Sprint Backlogs were maintained and organized in the project management tool of each team. Rally and RTC support the Scrum framework and user stories can be naturally added in the tools. In Redmine a ticket system was used to submit user stories and tasks. Each user story of the Product Backlog had a business value specified by the Product Owner. They were written following the pattern “As a <user> I should be able to <action> so that <business value>”. When adding to a sprint, the stories were decomposed into tasks and estimated based on developers’ experience using the poker game. Burn downs were generated at the end of Sprint by the teams. Rally and RTC generate the burn down charts automatically whereas Redmine does not. The Product Backlogs of the different project are comprised of 16 prioritized stories for the Android application, 20 prioritized user stories for the Blackberry application, and 32 prioritized stories for the Java ME application.

3.3. Quality assurance activities

Quality assurance included audits, testing and an awareness checkpoint that examined students’ awareness of their extended team and project’s status. Auditors performed audits of the Product Backlog, design, process, and code (in that order). Before engaging in their role, auditors attended lectures on software engineering process and played the XP games to develop an appreciation of Agile and Scrum practices. They provided definitions of quality and quality assurance, and refined them in terms of mobile application development. The definitions were shared with the development teams for the two parties to evolve on a common ground. An audit plan comprising audits of the Product Backlog, design, code and process was designed by the auditors. Audits were produced from the artifacts made available by the developers and testers. Reports consisted of the audit status (Green - compliant, Amber – minor issues, or Green – not compliant or not doing), a completed checklist to assess the quality of the artifact under audit, and constructive comments that address the strengths and the weaknesses of the work of the developers and actions to be taken. Auditors also reflected on the suitability of the tool used to support the software development process and project management and how it helped them in achieving their audit job. The developers had to read the audits, answer to the auditors, and incorporate the necessary feedback in a timely manner.

Audit of the Product Backlog. The checklists designed to audit the Product Backlog focused on confirming that the user stories were described at a high level, were written following the Scrum standard, had a business value provided by the customer, were prioritized, were divided into functional and non-functional stories, and comprised an acceptance test. Ambiguity, redundancy, amalgamation and conflict in user stories were also checked.

Audit of the Design. Elements of design, such as class diagrams, UI of the mobile applications, and organization of the data, were to be produced by the developers. The checklists designed to audit the design focused on verifying that design artifacts were produced, design options were traceable to the user stories or tasks, the UI followed standards related to the targeted platforms (e.g., consistency of the UI, and help and alerts for users) and addressed the constraints of the mobile platforms (e.g., size of the screen, battery life, and storage), and the developers implemented MVC.

Audit of the Process. The checklists designed to audit the process focused on ensuring that the Scrum roles were assigned and respected, the Scrum ceremonies were taking place and documented in due time, and the Scrum artifacts were produced and maintained. Particular

attention was directed to verifying that tasks were estimated, completed tasks were closed, and user stories committed during a sprint were implemented. Taking a holistic view, auditors checked that developers learned and improved based on comparing the velocity the developers committed to and the one they actually achieved. In addition, auditors checked that developers communicated regularly, addressed their comments, had some design available, used a coding standard, shared a code repository, and showed evidence of testing. The proficiency of the developers in using the project management tool was also evaluated.

Audit of the Code. The checklist to audit the code of the developers targeted coding style (e.g., comments and naming conventions for classes, methods and variables), modularity and readability of the code, exception handling, the extent of the use of MVC, the regular use of a code repository by all developers, and the amount of line of code of each developer.

4. Findings and recommendations

This section presents our project outcomes and answers our research questions.

4.1. Project outcomes

The Scrum Master logged the velocity and number of user stories implemented during each sprint by each team to visualize how the teams learned and improved. The data is presented in Table 2. Note that this data was not automatically generated from any of the project management tools used on this project. Developers overestimated the work they could achieve in sprint 2. Developers learned and improved their estimates in sprints 3 and 4, except one team that decided too optimistically to implement too many user stories in the last sprint. Some of the same issues were pointed out in the audits. The audits of the product backlog and process of the teams were Amber and the other audits were Green. Requirements wise, user stories were not always written following the Scrum pattern and were not complete; crucial non-functional stories specific to mobile applications were missing. Positively, all user stories had an associated business value and were prioritized. Process wise, developers did not estimate tasks and make reports of the meetings as regularly as they should. Developers reported that the Scrum framework was crucial for them as it gave them structure, organization and objectivity on the work they could achieve, especially as they were in a just-in-time learning environment. They were overwhelmed by the different activities and the discipline required by Agile and Scrum. In terms of design and coding, developers used coding conventions, and learned and integrated MVC proficiently.

Developers qualified the involvement of the auditors as “very helpful” to point out issues in the Product Backlog, process, design and coding that they needed to address in a timely manner. Using the developers’ words, auditors were particularly important in “keep[ing] them on track” with Agile and Scrum on the top of the guidance of the Scrum Master. They guided the process and permitted developers to have a better appreciation and adherence to Agile and Scrum because the audits were perceived as checkpoints.

Auditors were particularly enthusiastic about RTC and Rally to support their work. The strength of RTC is that it integrated the work of the developers and auditors using its wiki mechanism. In Rally, auditors had to use Google Groups for their reports as there is no wiki facility. This issue is addressed by a mechanism that permits to customize the Rally environment by adding external views that integrate external tools. The generation of a single document for the Product Backlog and the reporting facilities of Rally were particularly appreciated by the auditors and facilitated their work.

Table 2. Planned / actual implemented user stories (US) and velocity in each sprint (reported anonymously).

		Team1	Team2	Team3
Sprint 1	Planned / Actual Implemented US	9/7	7/6	4/3
	Planned / Actual Velocity	193/100	383/371	400/300
Sprint 2	Planned / Actual Implemented US	8/1	11/6	6/2
	Planned / Actual Velocity	152/49	215/131	392/140
Sprint 3	Planned / Actual Implemented US	8/8	7/4	5/5
	Planned / Actual Velocity	132/132	173/100	159/159
Sprint 4	Planned / Actual Implemented US	5/4	4/3	2/2
	Planned / Actual Velocity	142/94	173/160	80/80

4.2. Recommendations and benefits

Table 3 presents some recommendations related to communication, learning, training, roles and tooling for a better involvement of auditors in Agile and Scrum projects. The table also highlights the expected benefits of focusing on these aspects of the project.

5. Conclusion and future work

This paper described a global software development model that involved developers following Agile and Scrum to develop mobile applications for some popular mobile platforms. It was a first attempt to introduce auditors to provide additional support in fostering a process appreciation and adherence to Agile and Scrum. The work of the auditors was better supported by end-to-end tooling infrastructures such as RTC and Rally. Developers were satisfied by the mobile applications they developed even if they did not implement all the user stories they planned to because they found their experience in the project rewarding and challenging.

The 2011 GSD project will integrate the recommendations and lessons learned in 2010 to involve external Product Owners to increase the visibility of the interaction between the developers and the Product Owner. It will further seek to compare the use of RTC and Rally Software by developers and auditors in a more systematic manner.

6. Acknowledgement

This work was supported by an IBM Jazz Innovation Award “Evolving a Tooling Infrastructure for Global Software Development Projects”. We thank the 43 students and 3 instructors involved in the project: Vidya Kulkarni, Jean-Marie Preira and James Tamgno. We are grateful to Olly Gotel for previous work in introducing auditors in global software development project using a waterfall-like process.

Table 3. Integrating auditing activities in Agile and Scrum projects.

Findings	Recommendations	Benefits
Communications between developers and auditors	• Facilitate meetings between auditors and developers for auditors to publicize their audit plan and definition of quality.. • Add interviews with the developers to complement or replace some of the audits.	• Discover issues and the reasons of these issues, and create a trustful relationship between auditors and developers.
Auditing “Learn and Improve”	• Maintain an aggregated table (like Table 2) as part of a more complete way to assess the “learn and improve” in Scrum. • Proceed with a process audit every 2 sprints to report on how the team learned and improved.	• Better understanding of the effectiveness of the team.
Training auditors	• Play the XP game [9] as an introduction to Agile planning in a non technical context and add the Elephant Carpaccio exercise [3] to provide a technical perspective.	• The more familiar the auditors are with Agile and Scrum, the more thorough their audits will be.
Roles	• Have a Product Owner independent from the developers. • Introduce auditors and testers early in the process.	• Monitor the dynamics between the developers and the product owner and have more trust when stories are marked as “done”. • Improve the quality of the developed product.
Tooling	• Choose a tool supporting Agile and Scrum that can be customized for quality assurance activities and can integrate audits’ reports seamlessly. • Understand how the tool supports the different audits and the documentation of the evidence to be used for audits.	• More time spend on audits rather than learning tools and searching for the artifacts to audit.

7. References

- [1] Ågerfalk, P. J., Fitzgerald, B., Holmström Olsson, H., and Conchúir, E. O. “Benefits of Global Software Development: The Known and Unknown”. Lecture Notes in Computer Science, Springer Berlin, Volume 5007/2008, Making Globally Distributed Software Development a Success Story, p 1-9.
- [2] Ballard, B. 2007. Designing the Mobile User Experience. WileyBlackwel.
- [3] Cockburn, A. Elephant Carpaccio Exercise. <http://alistair.cockburn.us/Elephant+carpaccio>.
- [4] Deemer, P. and Benefield, G. The Scrum Primer: An Introduction to Agile Project Management with Scrum. Accessible at: <http://www.goodagile.com/scrumprimer/scrumprimer.pdf>.
- [5] Gotel, O., Kulkarni, V., Neak, L. and Scharff, C. "Students as Partners and Students as Mentors: An Educational Model for Quality Assurance in Global Software Development", Proc. of the Conf. on Soft. Engineering Approaches for Offshore and Outsourced Development (SEAFOOD), Zurich, Switzerland, 2008.
- [6] Hazzan, O. and Dubinsky, Y. "Teaching a Software Development Methodology: The Case of Extreme Programming". Proc. Of the International Conference on Software Engineering Education and Training (CSEET), Madrid, Spain, March 20-22, 2003.
- [7] Herbsleb, J.D. “Global Software Engineering: The Future of Socio-technical Coordination”. Proc. ICSE’07, The Future of Software Engineering (ICSE-FASE’07), Minneapolis, USA, 2007.
- [8] Manifesto for Agile Software Development. <http://agilemanifesto.org>.
- [9] Peters, V. and Van Cauwenberghe, P. The XP Game. <http://www.xp.be/xpgame.html> (Last access Dec. 2010).
- [10] Scharff, Gotel, and Kulkarni. "Transitioning to Distributed Development in Students’ Global Software Development Projects: The Role of Agile Methodologies and End-to-End Tooling", Proc. of the International Conference on Software Engineering Advances (ICSEA 2010), Nice, August 22 - 25, 2010.
- [11] Schwaber, K. and Beedle, M.A. Agile Software Development with Scrum. Pearson Education, 2008.
- [12] Wiki of the GSD 2010 project. <http://atlantis.seidenberg.pace.edu/wiki/gsd2010>.
- [13] Williams, L. Lessons Learned from Seven Years of Pair Programming at North Carolina State University. SIGCSE Bulletin. 39, 4 (Dec. 2007), 79-83.